Podium: An Open-Source Library for Rendezvous and Docking Guidance, Navigation, and Control with Integrated Formal Verification

Adi Oltean*

July 2026

Abstract

Podium is an open-source Python library for developing, testing, and formally validating guidance, navigation, and control (GNC) algorithms for spacecraft rendezvous, proximity operations, and docking (RPOD) in low and medium Earth orbit. It provides relative-motion models, convex and sequential-convex trajectory planners, a relative-navigation filter and controllers, a deterministic simulator, and a verification layer. A set of flight kernels—the relative-motion, navigation, and quaternion routines—is written in a restricted, statically analyzable Python subset, translated to C, shown alarm-free on contracted (or explicitly assumed) input ranges by the Frama-C Evolved Value Analysis (EVA) plug-in, and compiled with the CompCert verified compiler, with the emitted C checked against the Python reference on golden vectors. The barrier, KKT, and optimality-gap guarantees are re-verified in exact rational arithmetic in the trusted checker: tolerance-free barrier certificates for abort safety, Karush–Kuhn–Tucker (KKT) certificates that, when a valid Lagrangian dual point exists, give an exact rational bound on the suboptimality of an approximate online convex solve, and tolerance-free bounds on the global optimum of a nonconvex keep-out quadratic program representative of the docking geometry. This paper describes the architecture, the verification approach, and a reproducible end-to-end example.

1 Motivation and significance

Spacecraft rendezvous and docking place strong demands on software [1]: the maneuvers are safety-critical, some unsafe commands or contact events cannot be corrected after the fact, and the flight code must be shown to be free of runtime errors and faithful to its specification. Mature simulation environments exist, including Basilisk [2], NASA's 42 [3], Orekit [4], and the General Mission Analysis Tool (GMAT) [5], and Podium does not attempt to replace them for general astrodynamics. Its purpose is narrower: to organize development around spacecraft rendezvous, proximity operations, and docking (RPOD) rather than absolute orbit propagation, and to make formal validation part of ordinary development.

The library is motivated by three needs. First, relative motion, approach corridors, keep-out zones, plume impingement, and docking contact are central to RPOD and are a modeling focus distinct from absolute orbit propagation. Second, convex trajectory optimization is now standard in guidance [6, 7, 8, 9], but a floating-point quadratic program (QP) or second-order cone

*Starcloud, Inc. Source, documentation, and the companion technical notes referenced below (on the exact-arithmetic certificates and on numerical reproducibility) are in the repository under `docs/`: `github.com/adi-oltean/podium`, archived at `https://doi.org/10.5281/zenodo.21225268`.

program (SOCP) solver can return a nominally optimal solution that violates a constraint under rounding, so Podium re-checks such solver output with an exact-rational Karush–Kuhn–Tucker (KKT) checker that, when a valid dual point exists, computes an exact rational bound on the solution's suboptimality. Third, the gap between a Python prototype and analyzable flight-target C, ordinarily crossed by hand, is here crossed mechanically, so that validation evidence applies to the same code path intended for deployment.

2 Software description

2.1 Architecture

Figure 1 shows the two paths this description follows: the restricted Python-to-C flight-code path (top) and the exact-rational certificate layer (bottom). Podium is organized as subpackages under `src/podium`. The `core` package holds the verifiable static subset: the Clohessy–Wiltshire dynamics and state-transition matrix [10], the Yamanaka–Ankersen matrix for elliptic orbits [11], relative orbital elements with Keplerian, J_2, and J_2-plus-drag state-transition matrices [12], a quaternion kernel, and fixed-step integrators. The `dynamics` package holds the truth models: nonlinear relative motion with J_2 and differential drag over a seeded stochastic atmosphere, and rigid-body attitude with a suite of environmental torques (gravity gradient, aerodynamic, solar radiation pressure, and magnetic).

The `guidance` package implements convex and sequential-convex trajectory planners on the relative-motion discretizations. The `control` package provides discrete and continuous linear-quadratic regulators, quaternion attitude feedback, and a push-only thruster allocation. The `nav` package implements a relative-navigation extended Kalman filter with a Joseph-form (numerically stable) covariance update and sensor models for relative Global Navigation Satellite System (GNSS) navigation, camera, and lidar. The `sim` package provides a deterministic fixed-step engine with bit-identical replays, Signal Temporal Logic (STL) specification oracles, and contact checking. The `verify` and `emit` packages provide the verification layer and the C code generator described below.

2.2 The verifiable flight kernels

The flight kernels are written as pure step functions in a restricted subset of Python: statically shaped arrays, bounded loops, no dynamic allocation, and machine-readable contracts on input ranges and invariants. The subset is chosen so that a sound static analyzer can reason about it and so that its scalar and subscript statements translate line-for-line to C, with array expressions lowered to bounded row-major loops. The `emit` package generates C99 from the subset, annotated with ACSL (ANSI/ISO C Specification Language) contracts, with an optional CORE-MATH [13] backend for correctly rounded sine and cosine. The Frama-C/EVA abstract-interpretation analyzer [14] shows the emitted C free of runtime alarms (division by zero, overflow, invalid access) on the contracted or assumed input ranges. The generated C is checked against the Python reference on a suite of golden vectors—regenerated deterministically from a fixed per-kernel seed rather than stored as fixtures, bit-exact for the scalar arithmetic and square-root class, with documented tolerance classes for the libm transcendentals and for matrix products (which differ only by floating-point reassociation against NumPy's Basic Linear Algebra Subprograms)—and the same vectors are replayed through the CompCert verified compiler [15] on x86-64 and reproduced bit-for-bit on a second instruction-set architecture (ISA), AArch64. The golden vectors thus tie the Python reference, the emitted C, and the compiled assembly on the tested inputs; the reproducibility methodology, the equality classes, and the cross-ISA conditions (IEEE-754 binary64 with `-ffp-contract=off`) are documented in a

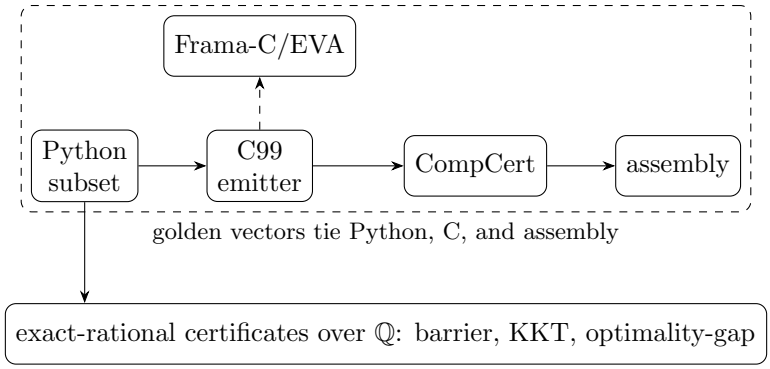

Figure 1: The flight-code path (top): a restricted Python subset is emitted to C99, shown alarm-free by Frama-C/EVA, and compiled by CompCert, with golden vectors tying the Python, C, and assembly on the tested inputs. A separate layer re-runs the exact-rational certificates (verified over $\mathbb{Q}$) together with reachability checks on the relevant path changes.

companion note on numerical reproducibility. This emitted, analyzed, and compiled path covers the flight kernels—the relative-motion state transitions, the navigation-filter update, and the quaternion routines—rather than the guidance planners or controllers, whose solver output the certificate layer re-checks below.

Software metadata. Language Python 3.10+; license MIT; NumPy required, with optional CVXPY, Clarabel, and ECOS for the solver-backed paths and optional verification tooling (Frama-C, CompCert, JuliaReach). Continuous integration (CI) runs on GitHub Actions. The examples and the per-release audit bundle are the primary distribution channel; the library is in active development (v0.x).

3 Verification approach

Podium runs verification as a set of automated checks re-executed on every relevant change, rather than as a manual milestone; Table 1 summarizes them.

Two implementation choices support these guarantees. First, several certificates are checked in exact rational arithmetic over the rationals $\mathbb{Q}$, with no floating point in the trusted checker: a barrier for abort safety [18, 19], KKT conditions for the online convex solves, and bounds on the global optimum of a nonconvex keep-out QCQP—a primitive of the docking geometry, not yet wired into the full-horizon planner—through the S-procedure [20, 21] and its trust-region special case [22]; the derivation, the hard (singular) case, and the multi-constraint extension are given in the companion note on exact certificates [23]. The barrier and optimality-gap certificates are tolerance-free, their acceptance being an exact positive semidefiniteness test together with exact feasibility; exact rational certificates of this kind build on a body of work in sum-of-squares and polynomial optimization and in verified semidefinite programming [24, 25, 26, 27, 28]. The same checker discharges further certificate families—control-Lyapunov and sum-of-squares certificates—whose constructions and exact-check basis are collected in the companion note on exact certificates. The KKT check re-verifies an approximate solver's output offline: it computes the optimality residuals exactly and, when a valid Lagrangian dual point exists, reports an exact rational bound on the point's suboptimality. For a strictly convex quadratic program this bound accounts for the stationarity residual through the problem's curvature; a small residual alone is not a suboptimality bound, since with a rank-deficient

Table 1: Automated verification checks, re-run in continuous integration on the relevant path changes (and on scheduled drift checks where noted). QP: quadratic program. SOCP: second-order cone program. QCQP: quadratically constrained quadratic program. ISA: instruction-set architecture.

Check	Guarantee
Closed-loop reachability (JuliaReach [16])	Flowpipes avoid unsafe sets on the benchmark models [17]
Exact-rational barrier certificates	Infinite-horizon abort safety, verified over $\mathbb{Q}$
Exact-rational KKT certificates	Online QP solves yield an exact rational suboptimality bound; SOCP solves re-verified against conic KKT; both over $\mathbb{Q}$
Exact optimality-gap certificates	Bounds on a nonconvex keep-out QCQP optimum, verified over $\mathbb{Q}$
Sound static analysis (Frama-C/EVA)	Emitted C alarm-free on contracted or assumed input ranges
Verified compiler and cross-ISA	Golden vectors preserved through CompCert and a second ISA
Independent physics oracle	Translational truth model cross-validated against Orekit [4]

objective (a linear program or minimum-fuel problem) an approximately stationary point can be arbitrarily suboptimal. For a second-order cone program, whose objective is linear, a rigorous bound instead requires exact conic-dual feasibility. The numerical solvers and the rational recovery that propose each certificate are untrusted; the trusted checker rationalizes the candidate and either verifies it exactly or refuses, so solver rounding can force a refusal or a weaker bound but never a false certificate. In every case the checker itself introduces no rounding error. Second, the emitted flight-code C is tied to the Python reference by the golden vectors (above) and carried to assembly by CompCert, so the validated algorithm and the compiled algorithm agree on the tested inputs. Every tagged release ships a byte-deterministic audit bundle: a build script gates the mission capture, the temporal-logic and docking-interface margins, and the barrier certificate, and emits the bundle alongside the generated flight C, a SHA-256 manifest over the byte-deterministic files, and the recorded package versions; the release additionally attaches a fresh Frama-C/EVA report, so that an identical-source rebuild is independently verifiable.

Figure 2 and Table 2 make these certificates concrete on the keep-out QCQP primitive: a quadratic objective (Hessian P_0) minimized subject to one quadratic keep-out constraint (Hessian P_1), whose S-procedure lower bound $g(\lambda)$ is built from the Lagrangian matrix $A(\lambda) = P_0 - \lambda P_1$ and brackets the global optimum $J^\star$. As the rational dual multiplier λ is rounded toward its optimum $\lambda^\star$, the exactly certified lower bound $g(\lambda)$ approaches the global optimum $J^\star$ at the rate the S-procedure predicts: quadratically when the trust-region subproblem is nonsingular, and linearly in the hard case where $A(\lambda^\star) = P_0 - \lambda^\star P_1$ is rank-deficient. Every plotted value is an exact rational verified by the checker. The KKT check behaves analogously (Table 2b): it returns an exact rational suboptimality bound only when a valid dual point exists, and refuses otherwise. A tolerance-based test would accept the rank-deficient point—stationarity residual 10^{-10}, yet 100 from the optimum—as optimal; the exact checker reports no bound rather than a false certificate.

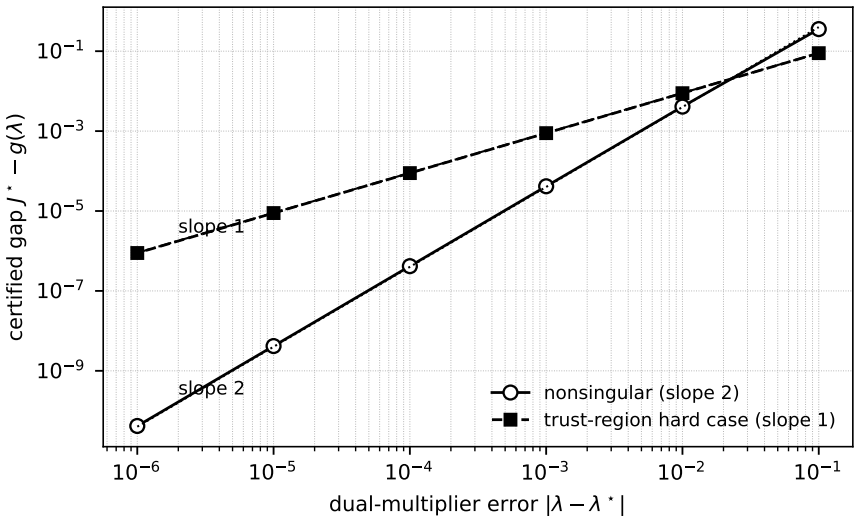

Figure 2: Exact optimality-gap recovery for a nonconvex keep-out QCQP. As the rational dual multiplier λ approaches its optimum $\lambda^\star$, the gap between the certified lower bound and the global optimum, $J^\star - g(\lambda)$, closes quadratically (slope 2) when $A(\lambda^\star)$ is nonsingular and linearly (slope 1) in the trust-region hard case. Both coordinates are exact rationals verified over $\mathbb{Q}$; the guides mark slopes 2 and 1.

Table 2: Exact-rational certificate receipts, verified over $\mathbb{Q}$ by the trusted checker. (a) Optimality-gap recovery for the keep-out QCQP of Fig. 2. (b) The KKT check returns an exact suboptimality bound only when a valid dual point exists; exact residuals alone are not an optimality certificate.

(a) Optimality-gap bracket recovery

Case	λ	$\lvert\lambda - \lambda^\star\rvert$	$J^\star - g(\lambda)$
Nonsingular	39/100	1/100	1/244
$\lambda^\star$=2/5, $J^\star$=4	399/1000	1/1000	1/24040
	3999/10000	1/10000	1/2400400
Hard case	101/100	1/100	803/90300
$\lambda^\star$=1, $J^\star$=−4/3	1001/1000	1/1000	8003/9003000

(b) KKT check: exact bound or refusal

Instance	Stationarity	Complementarity	Reported bound
PD QP $\frac{1}{2}x^2$, x=1	1	0	1/2 (certified)
Rank-deficient LP, x=0	10^{-10}	0	— (refused)
Indefinite $-\frac{1}{2}x^2$, x=0	0	0	— (refused)

4 Illustrative example

The reference mission (`podium.sim.mission`, exercised by `test_mission.py` in continuous integration) composes the layers end to end and is the paper's worked example. A penalized trust-region (PTR) sequential-convex planner produces a closed-loop rendezvous approach from the barrier-certified start set to the docking target; the plan is propagated through the nonlinear relative-motion truth model, and the resulting closed-loop trajectory (Figure 3) is checked against range-channel Signal Temporal Logic phase specifications. The passive-drift abort safety of the start set (the safe free-drift trajectory after a thruster cutoff) is certified in exact rational arithmetic by the barrier—an invariant of the linear Clohessy–Wiltshire drift for a fixed safe formation, which bounds the linearized abort. Richer truth forces (J_2, differential drag) and additional corridor/keep-out specifications are supported but are not enabled in this reference configuration. The exact-rational KKT and conic KKT checkers re-verify an online QP or SOCP solve against its own optimality conditions—for a second-order cone program, an embedded conic solver (ECOS) when it is installed. They run in continuous integration as part of the verification suite (`test_kkt.py`) rather than inside the reference-mission loop.

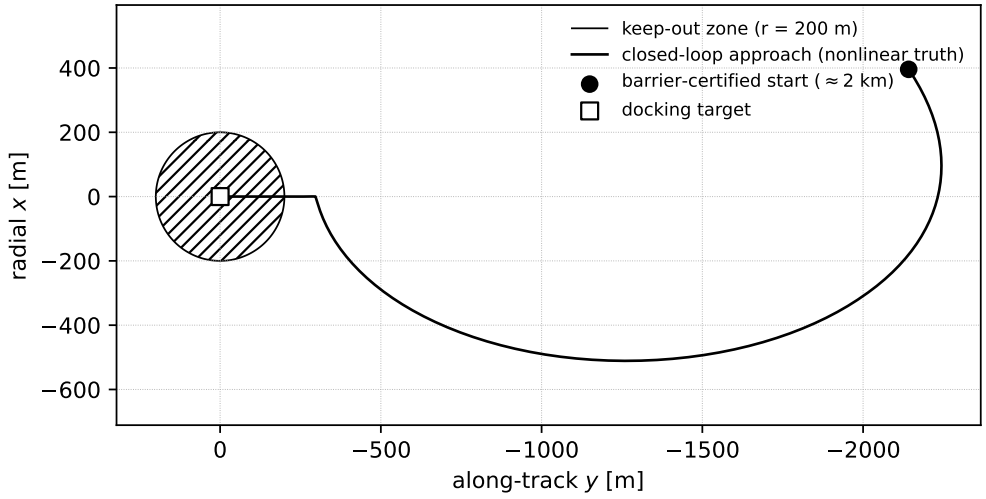

Figure 3: Reference mission: the closed-loop rendezvous approach from the barrier-certified start set (≈2 km) to the docking target under the nonlinear relative-motion truth model, in the local vertical/local horizontal (LVLH) frame. The far-field transfer keeps clear of the keep-out zone, and the terminal phase approaches along the docking corridor to contact; the passive-abort safety of the start set is certified in exact rational arithmetic by the barrier.

For a simpler entry point, the repository includes a one-command V-bar glideslope demo (`examples/vbar_approach.py`) with an interactive web playback, and a NASA Core Flight System (cFS) example application exercising the emitted kernels.

5 Impact and comparison

Each ingredient of Podium is well covered by open prior art; the contribution is their coupling. Table 3 makes this precise against the closest tools on each axis.

Table 3: Feature coverage of representative open tools: each covers at most one or two of the three prongs Podium combines. ✓ full, ∼ partial, blank none. Prong (i): a sequential convex programming (SCP) planner; (ii): exact-rational per-solve certificates re-run in continuous integration; (iii): a restricted high-level-source-to-C path checked by golden vectors, Frama-C/EVA, and CompCert. RPOD marks a rendezvous/docking focus.

	(i) SCP	(ii) cert/CI	(iii) verified C	RPOD
SCPToolbox.jl [29]	✓			∼
GuSTO.jl [30]	✓			∼
ct-scvx [31]	✓		∼	∼
successive_rendezvous [32]	✓			✓
RealCertify [33]		∼		
ValidSDP [34]		∼		
Verified SDP [35]		∼		
Copilot [36]			✓	
Credible autocoding [37, 38]			∼	
CVXPYgen [39]			∼	
Basilisk [2]				✓
Podium	✓	✓	✓	✓

On trajectory optimization, the open SCPToolbox.jl [29] and GuSTO.jl [30] implement the sequential-convex family with rendezvous and free-flyer examples; ct-scvx [31] adds continuous-time feasibility with in-house C generation, and successive_rendezvous [32] is a released six-degree-of-freedom Apollo transposition-and-docking implementation. Prong (i) for RPOD is thus established open art, and none of these released tools couples a certificate or verified-code layer. On certificates, RealCertify [33], the Coq-checked ValidSDP [34], and Verified SDP [35] produce exact-rational or rigorously bounded certificates for general polynomial and conic problems, and certifiable trajectory optimization [40], certifiable perception [41], and exact SDP relaxations for QCQPs [42, 43] produce certified bounds by numerical semidefinite programming; but none is online, RPOD-specific, tolerance-free over $\mathbb{Q}$, and re-run in continuous integration. On verified code, the Copilot toolchain [36] couples a restricted high-level source with a C path, formal proof, and CI (for runtime monitors, not guidance); credible autocoding [37, 38] and verified model predictive control solvers [44] autocode convex-optimization solvers to contract-annotated C, and execution-time-certified embedded QP solvers [45] bound the online iteration count; verified spacecraft control programs [46] and verified optimization reductions in a proof assistant [47] certify code or problem transformations formally; and CVXPYgen [39] generates C from Python convex problems; none couples exact-rational per-solve certificates with an RPOD library. The simulators Basilisk [2] and NASA's 42 [3] and the flight-dynamics systems Orekit [4] and GMAT [5] support proximity-operations modeling but no planner, certificate, or verified-code layer.

The novelty is therefore not any single prong but the integration of all three: a publicly released, RPOD-focused library that, in one codebase, couples sequential-convex planners with exact-rational *per-solve* certificates re-run in continuous integration and a restricted Python-to-C flight-code path checked by golden vectors, Frama-C/EVA under stated input contracts, and CompCert over its supported subset. Podium is complementary to the tools above rather than a replacement. It is intended for researchers and engineers developing and validating RPOD guidance and control, and as a reproducible baseline: verification runs on every change and every release carries its evidence, so that results built on the library can be re-checked independently.

6 Limitations

The guarantees are deliberately scoped, and each is conditional on its assumptions: the sound-analysis result holds while inputs remain within the declared or assumed contract ranges, and the certificates hold for the stated models and problem data, so establishing at the system level that flight inputs stay within contract and that the models bound reality belongs to the surrounding assurance case rather than the library. The verified path establishes runtime-error freedom and faithful Python-to-C-to-assembly translation on the tested inputs, not functional correctness against an external specification—a logic error in the source is carried through faithfully—nor solver convergence, the per-solve certificates instead bounding or refusing a returned solution. The cross-architecture bit-exactness of the emitted C holds for the scalar arithmetic and square-root class, which is correctly rounded and hence bit-identical across conforming hardware under IEEE-754 binary64 with fused-multiply-add contraction disabled; correctly rounded sine and cosine are bit-exact in that mode, while the remaining libm transcendentals and matrix products fall in documented tolerance classes.

The abort-safety barrier is certified exactly for the linear Clohessy–Wiltshire drift of a fixed formation—the standard e/i-vector passive-safety analysis, here machine-checked over $\mathbb{Q}$—and bounds the linearized abort; the residual gap to the nonlinear, J_2-and-drag, actuator- and sensor-perturbed trajectory is not certified by the barrier but is exercised by the nonlinear closed-loop simulation and its temporal-logic checks, so exact synthesis on the linear model and high-fidelity nonlinear simulation are complementary rather than substitutes. The exact-rational checkers run offline in continuous integration, not in the flight loop: they run on representative and regression problem instances, and can re-check any solver output whose exact problem data and candidate duals are supplied, so that a refusal or a false bound is caught in development; the reference mission does not currently attach a KKT certificate to its planner solves. Their rational entries stay small on the per-primitive problems the checkers target and grow with problem size, which is why they are applied per primitive rather than to arbitrary problem sizes.

The verifiable subset is intentionally restricted to statically shaped arrays, bounded loops, and no dynamic allocation—the memory- and timing-determinism discipline of high-integrity flight software (MISRA-C, the JPL Power of Ten)—so the emitted kernels are allocation-free and of fixed, bounded complexity—necessary conditions for high-integrity flight software, though not alone sufficient for qualification; variable-horizon, adaptive, and event-triggered logic is deliberately excluded from this path and lives in the untrusted planner and controller layers. Finally, Podium produces verification *evidence*—sound-analysis and verified-compiler receipts, exact certificates, and golden-vector equivalence—of kinds recognized in high-assurance software practice, rather than a completed qualification under a standard such as DO-178C or ECSS; mapping that evidence onto a certification case, and benchmarking the generated kernels against production baselines, are outside the present scope.

7 Conclusions

Podium is an open library that organizes guidance, navigation, and control around RPOD scenarios from the outset and treats formal validation as a continuous activity. Its flight kernels, exact-arithmetic certificates, and verified flight-code path are designed so that validation evidence applies to the same code path intended for deployment, and so that its guarantees can be reproduced by any user. The positive semidefiniteness checks common to these certificates use an $O(n^3)$ symmetric $LDL^\top$ factorization, whose factors are the nonnegativity witness. Broadening mission coverage and

hardening the verification toolchain are the main directions of continued development.

References

[1] Wigbert Fehse. *Automated Rendezvous and Docking of Spacecraft*. Cambridge University Press, 2003.

[2] Kenneth Alcorn, Hanspeter Schaub, and Scott Piggott. Simulating attitude actuation options using the Basilisk astrodynamics software architecture. *Acta Astronautica*, 2020.

[3] Eric T. Stoneking. 42: A comprehensive general-purpose simulation of attitude and trajectory dynamics and control of multiple spacecraft. NASA Goddard Space Flight Center, open-source software, 2020. `https://github.com/ericstoneking/42`.

[4] Luc Maisonobe, Véronique Pommier, and Pascal Parraud. Orekit: An open source library for operational flight dynamics applications. In *4th International Conference on Astrodynamics Tools and Techniques (ICATT)*, 2010.

[5] NASA Goddard Space Flight Center. General mission analysis tool (GMAT). Open-source software, 2020. `https://gmat.gsfc.nasa.gov`.

[6] Behçet Açıkmeşe and Scott R. Ploen. Convex programming approach to powered descent guidance for Mars landing. *Journal of Guidance, Control, and Dynamics*, 30(5):1353–1366, 2007.

[7] Behçet Açıkmeşe and Lars Blackmore. Lossless convexification of a class of optimal control problems with non-convex control constraints. *Automatica*, 47(2):341–347, 2011.

[8] Michael Szmuk and Behçet Açıkmeşe. Successive convexification for 6-DoF Mars rocket powered landing with free-final-time. In *AIAA SciTech Forum*, 2018.

[9] Danylo Malyuta, Taylor P. Reynolds, Michael Szmuk, Thomas Lew, Riccardo Bonalli, Marco Pavone, and Behçet Açıkmeşe. Convex optimization for trajectory generation. *IEEE Control Systems Magazine*, 42(5):40–113, 2022.

[10] W. H. Clohessy and R. S. Wiltshire. Terminal guidance system for satellite rendezvous. *Journal of the Aerospace Sciences*, 27(9):653–658, 1960.

[11] Koji Yamanaka and Finn Ankersen. New state transition matrix for relative motion on an arbitrary elliptical orbit. *Journal of Guidance, Control, and Dynamics*, 25(1):60–66, 2002.

[12] Adam W. Koenig, Tommaso Guffanti, and Simone D'Amico. New state transition matrices for relative motion of spacecraft in perturbed orbits. *Journal of Guidance, Control, and Dynamics*, 40(7):1749–1768, 2017.

[13] Alexei Sibidanov, Paul Zimmermann, and Stéphane Glondu. The CORE-MATH project. In *29th IEEE Symposium on Computer Arithmetic (ARITH)*, 2022.

[14] Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-C: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015.

[15] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.

[16] Sergiy Bogomolov, Marcelo Forets, Goran Frehse, Kostiantyn Potomkin, and Christian Schilling. JuliaReach: A toolbox for set-based reachability. In *22nd ACM International Conference on Hybrid Systems: Computation and Control (HSCC)*, 2019.

[17] Matthias Althoff et al. ARCH-COMP category report: Continuous and hybrid systems with linear continuous dynamics. In *ARCH Workshop*, 2023.

[18] Stephen Prajna and Ali Jadbabaie. Safety verification of hybrid systems using barrier certificates. In *Hybrid Systems: Computation and Control (HSCC)*, 2004.

[19] Aaron D. Ames, Xiangru Xu, Jessy W. Grizzle, and Paulo Tabuada. Control barrier function based quadratic programs for safety critical systems. *IEEE Transactions on Automatic Control*, 62(8):3861–3876, 2017.

[20] V. A. Yakubovich. S-procedure in nonlinear control theory. *Vestnik Leningrad University*, 1:62–77, 1971.

[21] Imre Pólik and Tamás Terlaky. A survey of the S-lemma. *SIAM Review*, 49(3):371–418, 2007.

[22] Jorge J. Moré and D. C. Sorensen. Computing a trust region step. *SIAM Journal on Scientific and Statistical Computing*, 4(3):553–572, 1983.

[23] Adi Oltean. Exact-rational certificates in Podium: Constructions, proofs, and prior art. Technical note, Starcloud, Inc., 2026. `https://doi.org/10.5281/zenodo.21247381`.

[24] Pablo A. Parrilo. Semidefinite programming relaxations for semialgebraic problems. *Mathematical Programming, Series B*, 96(2):293–320, 2003.

[25] Jean B. Lasserre. Global optimization with polynomials and the problem of moments. *SIAM Journal on Optimization*, 11(3):796–817, 2001.

[26] Erich L. Kaltofen, Bin Li, Zhengfeng Yang, and Lihong Zhi. Exact certification in global polynomial optimization via sums-of-squares of rational functions with rational coefficients. *Journal of Symbolic Computation*, 47(1):1–15, 2012.

[27] Helfried Peyrl and Pablo A. Parrilo. Computing sum of squares decompositions with rational coefficients. *Theoretical Computer Science*, 409(2):269–281, 2008.

[28] Maria M. Davis and Dávid Papp. Dual certificates and efficient rational sum-of-squares decompositions for polynomial optimization over compact sets. *SIAM Journal on Optimization*, 32(4):2461–2492, 2022.

[29] University of Washington Autonomous Controls Laboratory. SCPToolbox.jl: Sequential convex programming toolbox. Open-source software, 2022. `https://github.com/UW-ACL/SCPToolbox.jl`; companion to [9].

[30] Riccardo Bonalli, Abhishek Cauligi, Andrew Bylard, and Marco Pavone. GuSTO: Guaranteed sequential trajectory optimization via sequential convex programming. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2019. `https://github.com/StanfordASL/GuSTO.jl`.

[31] Purnanand Elango, Dayou Luo, Abhinav G. Kamath, Samet Uzun, Taewan Kim, and Behçet Açıkmeşe. Continuous-time successive convexification for constrained trajectory optimization. *Automatica*, 180:112464, 2025.

[32] Danylo Malyuta, Taylor P. Reynolds, Michael Szmuk, and Behçet Açıkmeşe. Six-degree-of-freedom rendezvous trajectory generation via successive convexification with state-triggered constraints. Open-source research code, AIAA SciTech, 2020. `https://github.com/dmalyuta/successive_rendezvous`.

[33] Victor Magron and Mohab Safey El Din. RealCertify: A Maple package for certifying non-negativity. *ACM Communications in Computer Algebra*, 52(2):34–37, 2018.

[34] Érik Martin-Dorel and Pierre Roux. ValidSDP: A Coq library for verified SDP-based positivity certificates. Coq library, 2017. `https://sourcesup.renater.fr/www/validsdp`.

[35] Christian Jansson. On verified numerical computations in convex programming. *Japan Journal of Industrial and Applied Mathematics*, 26:337–363, 2009.

[36] Ryan G. Scott, Mike Dodds, Ivan Perez, Alwyn E. Goodloe, and Robert Dockins. Trustworthy runtime verification via bisimulation (experience report). *Proceedings of the ACM on Programming Languages*, 7(ICFP):305–321, 2023.

[37] Timothy Wang, Romain Jobredeaux, Heber Herencia-Zapana, Pierre-Loïc Garoche, Arnaud Dieumegard, Eric Feron, and Marc Pantel. From design to implementation: An automated, credible autocoding chain for control systems. In *Advances in Control System Technology for Aerospace Applications*, volume 460 of *Lecture Notes in Control and Information Sciences*. Springer, 2016. arXiv:1307.2641.

[38] Raphaël Cohen, Eric Feron, and Pierre-Loïc Garoche. Verification and validation of convex optimization algorithms for model predictive control. *Journal of Aerospace Information Systems*, 17(5), 2020. arXiv:2005.12588.

[39] Maximilian Schaller, Goran Banjac, Steven Diamond, Akshay Agrawal, Bartolomeo Stellato, and Stephen Boyd. Embedded code generation with CVXPY. *IEEE Control Systems Letters*, 6:2653–2658, 2022.

[40] Shucheng Kang, Xiaoyang Xu, Jay Sarva, Ling Liang, and Heng Yang. Fast and certifiable trajectory optimization. arXiv:2406.05846, 2024.

[41] Heng Yang and Luca Carlone. Certifiably optimal outlier-robust geometric perception: Semidefinite relaxations and scalable global optimization. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(3):2816–2834, 2023.

[42] Masakazu Kojima, Sunyoung Kim, and Naohiko Arima. Local-to-global exactness of SDP relaxations for sparse QCQPs. arXiv:2606.21823, 2026.

[43] Diego Cifuentes, Sameer Agarwal, Pablo A. Parrilo, and Rekha R. Thomas. On the local stability of semidefinite relaxations. *Mathematical Programming*, 2022.

[44] Guillaume Davy, Eric Feron, Marc Pantel, Pierre-Loïc Garoche, and Didier Henrion. Experiments in verification of linear model predictive control. arXiv:1801.03833, 2018.

[45] Liang Wu, Wei Xiao, and Richard D. Braatz. EIQP: Execution-time-certified and infeasibility-detecting QP solver. arXiv:2502.07738, 2025.

[46] Andrey Mokhov, Georgy Lukyanov, and Jakob Lechner. Formal verification of spacecraft control programs using a metalanguage for state transformers. arXiv:1802.01738, 2018.

[47] Alexander Bentkamp, Ramón Fernández Mir, and Jeremy Avigad. Verified reductions for optimization. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2023.